koroteevmv / ML_course
Code
Issues 2
Pull requests
Actions
Projects
Security and quality
Insights
Files
main
Go to file
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
data
img
README.md
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.3 text
ML7.4 images
misc
ML_course / ML4.4 diagnostics /
Add file
History

koroteevmv Исправлены битые ссылки на картинки 3350f2e · last year

Name	Last commit message	Last commit date
..		
data	Картинки распределены по папкам	2 years ago
img	Картинки распределены по папкам	2 years ago
README.md	Исправлены битые ссылки на картинки	last year

README.md

Недообучение и переобучение

Цель работы

Познакомиться с основными проблемами обобщающей способности алгоритмов машинного обучения: overfitting (переобучение) и underfitting (недообучение).

Задания для выполнения

1. Загрузите первые 400 строк прилагающегося датасета `diabetes.csv`.
2. Сделайте количественное описание датасета: число признаков, статистику по признакам.
3. Отделите целевую переменную `Outcome`.
4. Разделите данные на обучающую и валидационную выборки при помощи `train_test_split` из библиотеки `sklearn.model_selection` в соотношении 80-20 (для этого укажите параметр test_size=0.2) с перемешиванием, указав значение параметра `random_state=42`.
5. Создайте объект `DecisionTreeClassifier(random_state=1)`. Обучите модель на обучающих (трейновых) данных. Сделайте предсказание на трейновом и валидационном наборе признаков. Выведите значения метрики `f1-score` для трейнового и валидационного наборов данных. По полученным значениям метрик сделайте предположение о переобученности модели.
6. Произведите кросс-валидацию с использованием функции `cross_validate` из библиотеки `sklearn.model_selection`. По полученным данным, постройте график зависимости значений `f1-score` от набора данных соответствующей итерации. По графику убедитесь в том, что имеет место переобученность модели.
7. Для борьбы с переобучением регуляризируйте модель `DecisionTreeClassifier`, уменьшив глубину дерева, указав параметр регуляризации `max_depth=3`.
8. Снова проделайте пункт 6 с учётом регуляризации и убедитесь по графику в том, что модель больше не является переобученной.
9. Теперь рассмотрите проблему недообучения модели. Для борьбы с недообучением модели добавьте данные. Для этого загрузите полностью датасета `diabetes.csv`.
10. Обучите модель `DecisionTreeClassifier(random_state=1, max_depth=3)` на всех данных и убедитесь в том, что значение метрики `f1-score` улучшилось.

Методические указания

Как всегда загружаем стандартный набор необходимых библиотек:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Диагностика моделей классификации

Для первого примера воспользуемся синтетическими данными. Сгенерируем большой и достаточно сложный датасет для классификации. В нем будет 10 тысяч точек и 500 атрибутов. Из них всего 50 признаков будут информативными. Сразу после генерации разделим получившийся датасет на обучающую и тестовую выборку:

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=10000, n_features=500,
                           n_informative=50, n_repeated=0,
                           class_sep=1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.25,
                                                    random_state=3)
```

Обучим на этом наборе данных самую простую модель - логистическую регрессию. Выведем оценки точности этой модели на обучающей и тестовой выборке:

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression().fit(X_train, y_train)

print(f"Training score: {lr.score(X_train, y_train):.4f}")
print(f"Test score: {lr.score(X_test, y_test):.4f}")
```

В данном случае мы используем метрику эффективности классификации по умолчанию. В зависимости от задания вы можете проводить анализ по любой интересующей вас метрике. Получаем такие оценки эффективности данной модели:

```
Training score: 0.8415
Test score: 0.8056
```

То, насколько отличается обучающая и тестовая эффективность модели - основной индикатор пере- или недообучения. Мы явно видим, что тестовая эффективность полученной модели невелика. Определить, что является причиной снижения качества модели - ее излишняя или недостаточная вариативность - основная цель диагностики моделей машинного обучения.

Но по одним этим показателям очень сложно понять, в какой из двух ситуаций мы находимся. Поэтому прибегнем к построению кривых обучения. Это даст нам больше информации о том, каков уровень сложности полученной модели по отношению к данным. Воспользуемся готовой реализацией построения кривых обучения из библиотеки продвинутой визуализации для машинного обучения *yellowbrick*:

```
from yellowbrick.model_selection import LearningCurve

visualizer = LearningCurve(
    LogisticRegression(), train_sizes=np.linspace(0.1, 1.0, 10)
).fit(X, y).show()
```

Здесь мы задаем модель, которую хотим проверить и набор долей обучающей выборки, по которым будем ее обучать. В данном случае, эта функция построит модели логистической регрессии, обучающиеся на 10%, 20% и так далее до 100% обучающей выборки. А затем по каждой модели будет вычислена обучающая и тестовая эффективность. И эти показатели будут изображены на графике, то есть кривая обучения:

Эта кривая показывает, что когда модель учится на небольшом объеме данных, она имеет прекрасную обучающую эффективность, но плохую тестовую. По мере увеличения объема обучающей выборки, эти эффективности сближаются, но между ними все еще сохраняется зазор.

Напомним, что большой промежуток между тестовой и обучающей эффективностью свидетельствует о переобучении. А низкое значение обучающей эффективности - о недообучении. Так что из этого присутствует в данной модели? Можно предположить, что модель немного переобучается. В пользу этого говорит и то, что исходный датасет очень разреженный (хотя мы об этом знаем только потому, что его сгенерировали). Более точно можно будет сказать сравнив эту модель с другими. Самый простой способ - использовать регуляризацию.

Мы воспользуемся классификатором на основе гребневой регрессии. Вспомним, что обычный метод наименьших квадратов (воплощением которого является модель логистической регрессии) эквивалентна гребневой модели с параметром регуляризации, равным нулю. Чем больше этот параметр, тем больше регуляризации в модели и тем проще полученная модель. Если мы предполагаем, что наша исходная модель переобучалась, то давайте построим и оценим модель с большим значением параметра регуляризации:

```
from sklearn.linear_model import RidgeClassifier
lr = RidgeClassifier(alpha=1000000).fit(X_train, y_train)

print(f"Training score: {lr.score(X_train, y_train):.4f}")
print(f"Test score: {lr.score(X_test, y_test):.4f}")
```

При таком огромном значении регуляризации мы почти гарантированно получим недообученную модель. Поэтому полезно будет сравнить результаты модели выше с этими. Вот что получилось у очень регуляризованной модели:

```
Linear Regression-Training set score: 0.7724
Linear Regression-Test set score: 0.7712
```

Обратите внимание, что эффективность стала заметно ниже. Плюс, значения почти совпадают. То есть, почти никакой разницы не осталось. Это подтверждает наш первоначальный вывод о недообучении. Так, что в первой модели разница между обучающей и тестовой эффективностью получилась довольно большой для данной задачи. Построим кривые обучения для нашей второй модели:

```
visualizer = LearningCurve(
    RidgeClassifier(alpha=1000000), train_sizes=np.linspace(0.1, 1.0, 10)
).fit(X, y).show()
```

Получилась совсем другая картина:

При анализе таких графиков надо обязательно обращать внимание на масштаб вертикальной оси. Несмотря на то, что кажется, будто значения в правой части графика (а нам интересны в первую очередь именно конечные значения эффективности, остальные - это просто предыстория) сильно больше, чем на первом, на самом деле они ниже. Данный график очень характерен для недообучения - низкие значения эффективности и практически никакой разницы между тестовой и обучающей эффективностью.

Если нерегуляризованная модель переобучается, а регуляризованная - недообучается, то где-то в промежутке есть оптимальная модель, котор

Для того, чтобы проанализировать данную задачу еще более глубоко построим график зависимости эффективности модели от значения параметра регуляризации. Для этого воспользуемся логарифмическим распределением:

```
from sklearn.linear_model import RidgeClassifier
trains = []
tests = []
for i in np.logspace(2, 6, 50):
  ridge = RidgeClassifier(alpha=i).fit(X_train, y_train)
  trains.append(ridge.score(X_train, y_train))
  tests.append(ridge.score(X_test, y_test))

plt.plot(trains)
plt.plot(tests)
```

Данный код построит 50 моделей, различающихся только параметром регуляризации. Он будет лежать в диапазоне от 100 (10^2) до 100 000 (10^6). Диапазон подбирается эмпирически, интересующий нас интервал значений параметра регуляризации будет разный в других задачах. Поэтому рекомендуется сначала построить модели в широких пределах (скажем, от -6 до 6), а затем сужать его настолько, чтобы на графике наглядно прорисовалась примерно такая картина:

Здесь мы видим, что при определенных значениях параметра регуляризации тестовая эффективность модели (а нас в конечном итоге интересует именно она) лучше, чем во всех остальных. Подбором или алгоритмически можно найти такое оптимальное значение параметра регуляризации. Оно даст нам модель, которая имеет оптимальный уровень сложности для данного конкретного датасета. Давайте выведем получившиеся показатели:

```
from sklearn.linear_model import RidgeClassifier
lr = RidgeClassifier(alpha=26500).fit(X_train, y_train)

print(f"Training score: {lr.score(X_train, y_train):.4f}")
print(f"Test score: {lr.score(X_test, y_test):.4f}")
```

Эта модель получилась немного лучше, чем самая первая, модель логистической регрессии.

```
Training score: 0.8311
Test score: 0.8228
```

Это произошло потому, что мы за счет регуляризации убрали негативный эффект переобучения. Давайте построим кривые обучения, чтобы посмотреть, как они выглядят для "хорошей" модели:

Опять же, с оглядкой на масштаб вертикальной оси мы видим, что и уровень эффективности выше, чем в первой модели и разница между двумя эффективностями тоже сократилась. И то и другое - это свидетельство большей обобщающей способности модели.

Диагностика моделей регрессии

Для более реалистичного примера загрузим датасет цен на недвижимость:

```
from numpy import genfromtxt
dataset = genfromtxt('https://raw.githubusercontent.com/m-mehdi/tutorials/main/boston_housing.csv', delimiter=',')
X = dataset[:,:-1]
y = dataset[:,-1]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)
```

Это набор данных для решения задачи регрессии - предсказания цен объектов недвижимости по их характеристикам. Самостоятельно изучите данный датасет, приведите его численные показатели. А мы сразу выведем оценку эффективности простой модели - линейной регрессии:

```
Linear Regression-Training set score: 0.95
Linear Regression-Test set score: 0.61
```

Большой разрыв между обучающей и тестовой эффективностью также сразу должен навести на мысль о переобучении данной модели. Попробуем проверить наше предположение при помощи кривых обучения (обратите внимание, что мы сразу используем требуемую модель с очень маленькой регуляризацией):

```
from yellowbrick.model_selection import LearningCurve
visualizer = LearningCurve(
    Ridge(alpha=0.0001), scoring='r2', train_sizes=np.linspace(0.7, 1.0, 10)
).fit(X, y) .show()
```

Может потребоваться несколько попыток, чтобы заставить графики выглядеть показательно (может понадобится изменить масштаб по разным осям), но в итоге мы можем наблюдать примерно такой график:

Обратите внимание, что промежуток между обучающей и тестовой эффективностью все равно очень большой в правой части графика, хоть часто визуально и не выглядит так. Просто тестовая эффективность в левой части графика просто ужасна, и она сильно смещает вертикальную ось вниз. Тем не менее, можно обоснованно предположить, что такая модель переобучается на имеющихся данных.

Сразу построим график зависимости эффективности модели от значения параметра L2-регуляризации:

```
from sklearn.linear_model import Ridge
trains = []
tests = []
for i in np.logspace(-2, 3, 100):
    ridge = Ridge(alpha=i).fit(X_train, y_train)
    trains.append(ridge.score(X_train, y_train))
    tests.append(ridge.score(X_test, y_test))

plt.plot(trains, label="train")
plt.plot(tests, label="test")
plt.legend()
```

Получаем такой очень характерный график регуляризации:

На этом графике очень наглядно виден характер кривых регуляризации. В левой области лежат значения, которые дают слишком сложные модели. Они хорошо описывают обучающую выборку, но не обобщаются на тестовую. Это зона переобучения. В правой области - значения, порождающие слишком простые модели. Они одинаково плохо описывают и обучающую и тестовые выборки. Это зона недообучения. Посередине между ними лежит оптимальный уровень сложности модели для данного конкретного датасета. Здесь эта оптимальность очень хорошо выражена.

Численно можно подобрать оптимальное значение параметра регуляризации для гребневой регрессии. Давайте выведем эффективность такой оптимальной модели:

```
lr = Ridge(alpha=0.19).fit(X_train, y_train)

print(f"Linear Regression-Training set score: {lr.score(X_train, y_train):.2f}")
print(f"Linear Regression-Test set score: {lr.score(X_test, y_test):.2f}")
```

Для такой модели получаем следующие значения:

```
Linear Regression-Training set score: 0.92
Linear Regression-Test set score: 0.77
```

Можно видеть, что они значительно выше, чем в первой модели. Опять же, мы оптимизировали модель за счет борьбы с ее переобучением. Давайте построим кривую обучения данной модели:

```
visualizer = LearningCurve(
    Ridge(alpha=0.19), scoring='r2', train_sizes=np.linspace(0.1, 1.0, 10)
).fit(X, y) .show()
```

Здесь же, вся разница между кривыми обучения лежит в масштабе вертикальной оси. По сравнению с предыдущим графиком этот выглядит гораздо более адекватно.

Здесь можно заметить, что, несмотря на то, что мы подобрали наилучший уровень сложности модели, ее эффективность все равно не идеальна. Присутствует и разница между обучающей и тестовой эффективностью. Это свидетельствует о том, что и в нашей самой лучшей модели все равно имеется и bias и variance в какой-то степени. И хоть мы нашли лучшую модель, она лучшая из своего отдельного класса моделей - гребневой регрессии.

Поэтому можно исследовать на этом же датасете и другие модели. Так, например, можно попробовать другой вид регуляризации - лассо. В библиотеке sklearn эта модель работает только как регрессор, то есть она не подходит для задач классификации. Поэтому мы не могли использовать ее в предыдущем примере. Сразу построим кривые регуляризации:

```
from sklearn.linear_model import Lasso
trains = []
tests = []
for i in np.logspace(-4, 0, 100):
    ridge = Lasso(alpha=i).fit(X_train, y_train)
    trains.append(ridge.score(X_train, y_train))
    tests.append(ridge.score(X_test, y_test))

plt.plot(trains, label="train")
plt.plot(tests, label="test")
plt.legend()
```

Получается такой график тоже с отчетливыми зонами пере- и недообучения:

Точно также можно исследовать и другие виды моделей. Итого для диагностики моделей используется три инструмента:

- сравнение обучающей и тестовой эффективности
- анализ формы кривых обучения
- построение кривых по одному из гиперпараметров модели, который влияет на соотношение bias/variance

Но главное - это сравнение между собой нескольких моделей одного типа, но с разным уровнем сложности (еще говорят про модели с разной склонностью к пере- или недообучению).

Диагностика на реальных данных

В реальных задачах диагностика моделей машинного обучения может быть не настолько очевидной. Зачастую используются модели, которые не используют регуляризацию в явном виде. В этом примере рассмотрим приемы диагностики моделей на реальных данных. Будем использовать известный датасет для классификации "Диабет"

Загружаем данные:

```
import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/koroteevmv/ML_course/2023/ML4.3%20diagnostics/diabetes.csv',nrows=200)
df.head()
```

Обратите внимание, что мы считали только первые 200 строк из файла. Этим мы имитируем обучение на малых данных. Формируем обучающую и тестовую выборки:

```
target = "Outcome"
y = df[target]
X = df.drop(target, axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Для моделирования выберем совершенно другой тип модели - решающие деревья. Создадим объект модели дерева решений и исследуем его эффективность:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(random_state=1).fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_pred = model.predict(X_test)
print("Train score = %.4f" % accuracy_score(y_train, y_train_pred))
print("Test score = %.4f" % accuracy_score(y_test, y_pred))
```

Получаем следующий очень показательный результат:

```
Train score = 1.0000
Test score = 0.5500
```

Даже без построения кривых обучения здесь очевидно переобучение модели. Но все же для тренировки распознавания разных ситуаций на графике построим кривую обучения:

У деревьев решений один из параметров - максимальная глубина дерева как раз и влияет на склонность модели к переобучению. Аналогом регуляризации для деревьев будет искусственное ограничение максимальной глубины дерева. Давайте построим кривую обучения для такого регуляризованного дерева:

```
visualizer = LearningCurve(
    DecisionTreeClassifier(max_depth=4), train_sizes=np.linspace(0.1, 1.0, 10)
).fit(X, y) .show()
```

Получается гораздо более качественная картина:

Выведя метрики эффективности получаем более высокое качество модели на тестовых данных:

```
Train f1_score = 0.8500
Test f1_score = 0.6250
```

Введение регуляризации, искусственное упрощение моделей - не единственный способ борьбы с переобучением. Самый лучший способ - это обучение на более полном датасете, то есть добавление большого количества данных. В этом примере мы специально не использовали весь имеющийся набор данных. На части модели сильно переобучается, но чем больше данных, тем меньше модели (даже одинаковые по сложности) становятся склонными к переобучению.

Для иллюстрации этого повторим обучение нерегуляризованного дерева решений на всем датасете:

```
df = pd.read_csv('https://raw.githubusercontent.com/koroteevmv/ML_course/2023/ML4.3%20diagnostics/diabetes.csv')
target = "Outcome"
y = df[target]
X = df.drop(target, axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(random_state=1).fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_pred = model.predict(X_test)
from sklearn.metrics import f1_score
print("Train score = %.4f" % accuracy_score(y_train, y_train_pred))
print("Test score = %.4f" % accuracy_score(y_test, y_pred))
```

В итоге мы получаем такие метрики эффективности:

```
Train score = 1.0000
Test score = 0.7273
```

Получившаяся модель значительно лучше, чем даже регуляризованное дерево. В общем случае, добавление данных работает лучше, чем регуляризация.

Диагностика недообучения

До сих пор мы рассматривали примеры, в которых модели по умолчанию демонстрируют явное переобучение. Но на практике чаще сталкиваются с недообучением, так как начинают анализ с более простых моделей, которые как раз склонны именно к недообучению. Рассмотрим в качестве примера более сложный датасет.

В sklearn есть встроенный механизм загрузки датасетов из онлайн репозитория OpenML. Возьмем оттуда датасет, посвященный проблеме управления истребителем F16:

```
from sklearn.datasets import fetch_openml
data = fetch_openml(name='delta_ailerons', parser="auto")
data.data.shape
```

Он состоит из более 7 тысяч строк:

```
(7129, 5)
```

Сформируем обучающую и тестовую выборки:

```
X = pd.get_dummies(data.data)
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Создадим и оценим простую модель логистической регрессии:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_pred = model.predict(X_test)
print("Train score = %.4f" % accuracy_score(y_train, y_train_pred))
print("Test score = %.4f" % accuracy_score(y_test, y_pred))
```

Получаем следующие метрики:

```
Train score = 0.6058
Test score = 0.5940
```

Построим кривую обучения данной модели:

```
visualizer = LearningCurve(
    LogisticRegression(), scoring='accuracy', train_sizes=np.linspace(0.3, 1.0, 10)
).fit(X, y) .show()
```

Здесь очевидны признаки недообучения: низкий уровень обучающей эффективности, малый разрыв между обучающей и тестовой эффективностью. При недообучении регуляризация не поможет, даже навредит, так как еще усилит его. Поэтому единственный путь в этом случае - использовать более сложные, вариативные модели. Можно, например, попробовать многослойный перцептрон:

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier().fit(X_train, y_train)
y_train_pred = model.predict(X_train)
y_pred = model.predict(X_test)
print("Train score = %.4f" % accuracy_score(y_train, y_train_pred))
print("Test score = %.4f" % accuracy_score(y_test, y_pred))
```

```
Train score = 0.9388
Test score = 0.9390
```

Самого главного мы добились - эффективность модели сильно выросла. Но давайте построим график обучения:

```
visualizer = LearningCurve(
    MLPClassifier(), scoring='accuracy', train_sizes=np.linspace(0.3, 1.0, 10)
).fit(X, y) .show()
```

Кроме использования более вариативных моделей самих по себе можно ввести в модель полиномиальные признаки:

```
from sklearn.linear_model import RidgeClassifier
from sklearn.preprocessing import PolynomialFeatures
model = RidgeClassifier(alpha=0.001).fit(PolynomialFeatures(2).fit_transform(X_train), y_train)
y_train_pred = model.predict(PolynomialFeatures(2).fit_transform(X_train))
y_pred = model.predict(PolynomialFeatures(2).fit_transform(X_test))
print("Train score = %.4f" % accuracy_score(y_train, y_train_pred))
print("Test score = %.4f" % accuracy_score(y_test, y_pred))
```

Полиномиальные модели являются естественным усложнением линейных и поэтому в случае недообучения исходной модели почти всегда обеспечивают увеличение эффективности модели:

```
Train score = 0.9392
Test score = 0.9362
```

Для этой полиномиальной модели уже можно пробовать использовать регуляризацию и строить соответствующие кривые:

```
from sklearn.linear_model import RidgeClassifier
trains = []
tests = []
for i in np.logspace(-7, 0, 100):
  ridge = RidgeClassifier(alpha=i).fit(PolynomialFeatures(2).fit_transform(X_train), y_train)
  trains.append(ridge.score(PolynomialFeatures(2).fit_transform(X_train), y_train))
  tests.append(ridge.score(PolynomialFeatures(2).fit_transform(X_test), y_test))

plt.plot(trains, label="train")
plt.plot(tests, label="test")
plt.legend()
```

В этом примере (как и во многих других реальных случаях) кривая регуляризации уже не такая очевидная и легко читаемая. Но и на ней можно при желании разглядеть область недообучения справа. Область переобучения здесь просматривается не так четко, так как тестовая эффективность практически не падает с уменьшением параметра регуляризации:

Также можно заметить, что этот график уже не такой гладкий и ровный, как в предыдущих примерах. На нем присутствуют скачки, неравномерности. Это все случайные отклонения, которые обусловлены и артефактами в самих данных, и стохастичностью процесса обучения сложных моделей и случайными ошибками выборки, возникшими при разделении датасета на тестовую и обучающую выборки.

Задания для самостоятельного выполнения

1. Повторите диагностику модели классификации синтетических данных с использованием других метрик эффективности - precision, recall, F1. Как изменились результаты?
2. Повторите диагностику модели регрессии с использованием других видов моделей:
   i. метода опорных векторов (исследуйте влияние параметра C на недо- и переобучение)
   ii. метода ближайших соседей (исследуйте влияние количества соседей)
   iii. метода решающих деревьев (гиперпараметры по вашему выбору)
3. Дополните анализ датасета Бостон исследованием модели лассо - постройте кривые обучения для переобученной, недообученной и качественной модели, найдите оптимальное значение параметра регуляризации в модели лассо.
4. В одном из двух первых примеров используйте модель ElasticNet и исследуйте влияние сразу двух параметров регуляризации на эффективность модели.
5. Дополните анализ дерева решений из третьего примера: постройте кривые регуляризации для полного датасета, найдите оптимальную глубину дерева и оцените качество такой модели на всем датасете. Повторите данный анализ используя в качестве метрики эффективности F1.
6. Повторите анализ на других реальных датасетах из репозитория OpenML:
   - phoneme
   - banknote-authentication
   - delta_ailerons
   - mammography
   - electricity
   - mozilla4
   - kropt
   - nursery
   - volcanoes-b3

Контрольные вопросы

1. Что такое обобщающая способность модели?
2. Что такое переобучение модели?
3. Какие есть способы борьбы с переобучением?
4. Что такое регуляризация модели, как она работает?
5. Чем отличаются разные методы регуляризации моделей?
6. Что такое недообучение модели?
7. Какие есть способы борьбы с недообучением?

Дополнительные задания

1. Повторите анализ последнего примера но с другими разделениями датасета на обучающую и тестовую выборки (изменяйте параметр random_state). Как меняется вид графиков?
2. Напишите код построения кривых регуляризации, использующий кросс-валидацию.